# Data Augmentation Method for Improving the Accuracy of Automated Test Case Generation Using Attention Seq2Seq Model

1st Taiga Ishinari
*Graduate School of Engineering*
*Nihon University*
Koriyama, Japan
ceta25002@g.nihon-u.ac.jp

2nd Kiyoshi Ueda
*College of Engineering*
*Nihon University*
Koriyama, Japan
ueda.kiyoshi@nihon-u.ac.jp

2026 IEEE 2nd International Conference on Consumer Technology (ICCT-Pacific) | 979-8-3315-5967-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICCT-PACIFIC69083.2026.11518865

***Abstract*—We aim to automate the development of communication software. By leveraging machine learning, we propose a method for automatically generating system testing test cases from requirement specification documents. In this research, we focus on developing a method to generate test cases by structuring requirement specification sentences using a deep learning Seq2Seq model with an Attention mechanism. To improve the accuracy of previous research, we propose a method that extends training data using natural language processing data augmentation. To evaluate the effectiveness and characteristics of the proposed method, we conducted experiments using various IT system requirement specification documents.**

***Index Terms*—test cases generation, data augmentation, attention seq2seq model.**

## I. Introduction

In the development of communication software, the waterfall model is widely used as the mainstream approach. Fig. 1 illustrates the concept of automatically generating test cases within the V-model framework. In the development of high-quality, large-scale communication software, improving efficiency and automating the extraction of system test cases from specifications has been a persistent challenge. To address this, we have been investigating a method for automatically generating system test cases from requirement specifications using machine learning [1]. To further enhance the accuracy of test cases generated with machine learning, we aim to develop a method for automatic test case generation by structuring requirement specifications using deep learning.

## II. Previous Research

### A. Automatic Test Case Generation Method

The method for automatically generating test cases is divided into four phases. The process flow of this method is illustrated in Fig. 2.

*1) Phase of Structuring the Requirements Specification (Tagging):* First, experts assigns tags to parts of the requirement specification corresponding to the test cases. These tags indicate input/output information, checkpoints for each function, and other details, thereby structuring the requirement specification (formulation of expert know-how). For example, in Fig. 3, the section describing the trigger for the action, "when the caller dials the time signal service number (117)," is structured by enclosing it within the <input>tag.

*2) Phase of Machine Learning:* The structured requirement specification is divided into morphemes (the smallest units of words that have meaning) using morphological analysis software (MeCab [2]), and parts of speech are assigned to create the training data. By dividing into morphemes, the tag information assigned to the sentence is assigned to each morpheme as each label. Fig. 4 shows an example of data

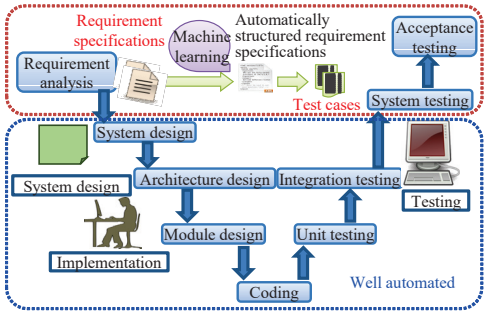


Fig. 1. Image of automatic generation of test cases in V-model.

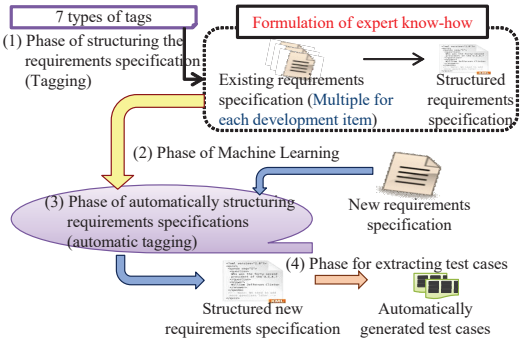


Fig. 2. Automatic test cases generation method.

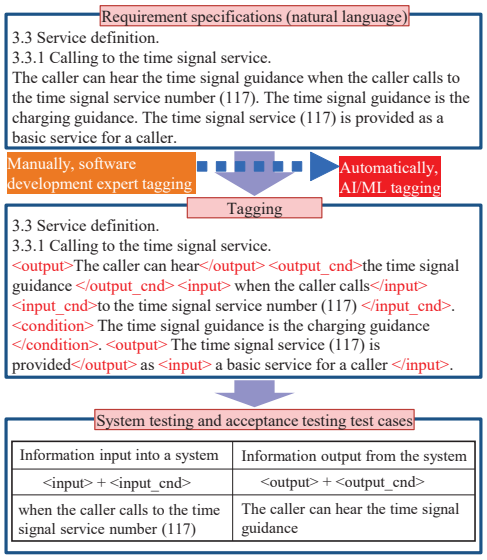


Fig. 3. Tagging for structuring requirements specification.

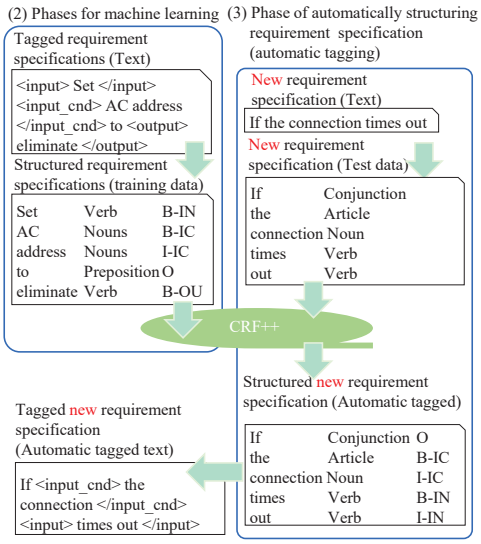


Fig. 4. Automatically structuring data processing.

processing according to the flow of the automatic test cases generation method. For example, the part enclosed in the <input_cnd>tag "AC address" is broken down into the morphemes "AC" and "address" by morphological analysis, and B-IC label is assigned to "AC" and I-IC label is assigned to "address". The training data with tags assigned to each morpheme is then used to train the machine learning Conditional Random Fields (CRF) tool CRF++ [3]. At this time, a learning model file is generated.

*3) Phase of Automatically Structuring Requirements Specifications (Automatic Tagging):* The new requirement specification is analyzed using morphological analysis. When the new requirement specification is input into CRF++, which has

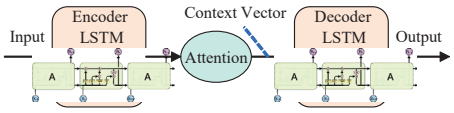


Fig. 5. Attention Seq2Seq model.

been trained using the model file generated in the previous phase, a new requirement specification with labels for each word is created.

*4) Phase for Extracting Test Cases:* The test cases are automatically extracted by a program that extracts the parts with tags indicating input and output information.

### *B. Evaluation Method for Automatic Test Case Generation*

The accuracy of the automatic structuring of the requirements specification is calculated by the $Precision$ and $Recall$ for each tag assigned to each word in the requirements specification. Then calculating the $F\text{-}measure$ (harmonic mean) as the accuracy.

### *C. Automatic Specification Structuring Method Using LSTM*

Neural Networks (NNs), inspired by human neural circuits, consist of a multi-layer architecture comprising an input layer, one or more hidden (intermediate) layers, and an output layer. Deep learning consists of multiple hidden layers. Long Short-Term Memory (LSTM), a type of Recurrent Neural Network (RNN) well-suited for learning time-series data, includes three gates in its hidden layer: a forget gate, an input gate, and an output gate. These gates control the flow of long-term and short-term memory, enabling the model to learn long-range dependencies. We employed LSTM model for machine learning to automatically structure requirement specifications [4].

### *D. Automatic Structuring Requirement Specifications Using Attention Seq2Seq Model*

In a conventional Seq2Seq model, time-series information is encoded into a fixed-length vector passed from the encoder to the decoder. However, the Attention mechanism enables the model to generate context vectors that selectively emphasize relevant parts of the input, thereby capturing the relationships between words more effectively, as illustrated in Fig. 5. The Transformer architecture, which underlies many modern generative AI models, incorporates the Attention mechanism into the Seq2Seq framework. We apply this Attention Seq2Seq model to the task of automatic test case generation [5].

### *E. Issues in Previous Research*

The Attention Seq2Seq model achieved higher accuracy than the LSTM model. However, we also observed an issue of overfitting. A possible cause is insufficient training data. Therefore, to resolve overfitting and further improve model accuracy, we propose a training data augmentation method based on data augmentation techniques which is useful in natural language processing.

## III. Proposed Method 1

### A. Method Using EDA

We apply Easy Data Augmentation (EDA) to structured specifications, that is considered an effective natural language data augmentation technique. EDA randomly selects one of the following four tasks with 25% task ratio each and executes it $n$ times. $n$ is determined by $n = \alpha \times N$. $N$ is the number of words in the original sentence. EDA adaptation ratio $\alpha(0 \leq \alpha \leq 1)$ is the probability of applying a task.

- Synonym Replacement (SR): Randomly choose words from the sentence that are not stop words. Replace each of these words with one of its synonyms chosen at random.
- Random Insertion (RI): Find a random synonym of a random word in the sentence that is not a stop word. Insert that synonym into a random position in the sentence.
- Random Swap (RS): Randomly choose two words in the sentence and swap their positions.
- Random Deletion (RD): Randomly remove each word in the sentence with probability $\alpha$.

Synonyms required for the SR and RI tasks are sourced from Japanese WordNet [6]. The label assigned to the expanded word remains the same as the original word. Words that appear frequently in the text but are not important (stop words) are excluded from the expansion process. In this method, Japanese particles and symbols were excluded as stop words.

### B. Evaluation of Proposed Method 1

*1) Evaluation Perspectives:* We applied Proposed Method 1 to 14 IT system specifications. The names of each specification and the number of sentences per specification are the same as those in Table I of Reference [5]. The same number of extended sentences as in the original text were generated. Both the original and the extended sentences were used as training data. We compared the accuracy of the Proposed Method 1 with that of the CRF, which demonstrated the highest accuracy in previous studies, and an Attention Seq2Seq model without data augmentation. In this experiment, we use Attention Seq2Seq model. The hyperparameters were set to the values that achieved the best results in our previous research: epoch $= 100$, initial learning rate $= 0.001$, and batch size $= 20$. In addition, the EDA adaptation rate $\alpha = 0.1$.

*2) Evaluation Conditions:* Excluding Japanese particles, symbols, and auxiliary verbs, we compare the accuracy after tag grouping as shown in Fig. 6, where B-IN and I-IN, B-IC and I-IC are replaced with IN, and B-OU and I-OU, B-OC and I-OC are replaced with OU. The accuracy after tag grouping refers to the accuracy of test case generation. One specification document was used as test data, while the other 13 were used as training data to create a model file and measure its accuracy. Each of the 14 specification documents was used as test data, and a model file was created using the remaining documents to measure its accuracy. The average and standard deviation were calculated.

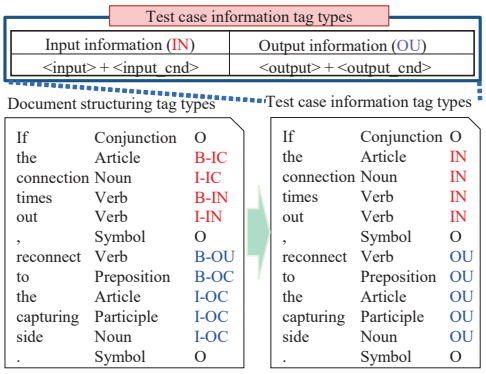


Fig. 6. Conversion to test case information tag types.

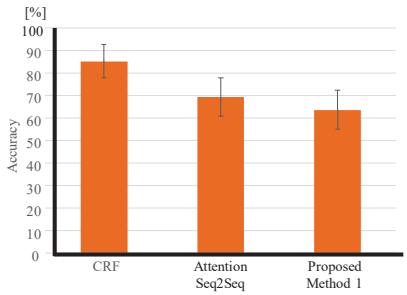


Fig. 7. Comparison of the mean and standard deviation of the accuracy of Proposed Method 1 with the conventional method.

*3) Evaluation Results:* Example of a generated SR is that the original sentence is "Output the application logs" and the extended sentence is "Response the application logs". As shown in Fig. 7, the accuracy of Proposal Method 1 was lower than in previous research.

This is thought to be caused by overfitting. Fig. 8 shows the transition in loss values over epochs for the training data and validation data. The loss value for the training data decreases smoothly, but the loss value for the validation data increases. The primary cause of overfitting is considered to be insufficient difference between the expanded text and the original text.

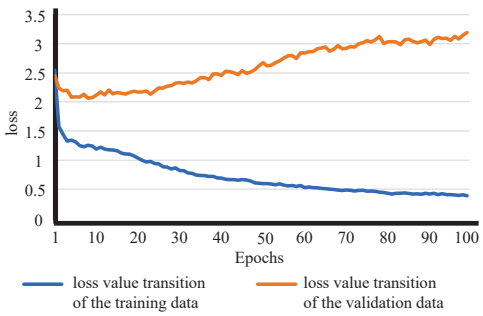


Fig. 8. Loss value for epochs in the Proposed Method 1.

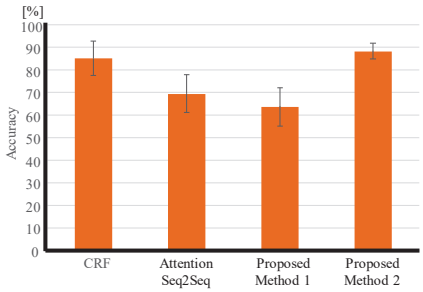


Fig. 9. Comparison of the mean and standard deviation of the accuracy of Proposed Method 2 with the conventional method.

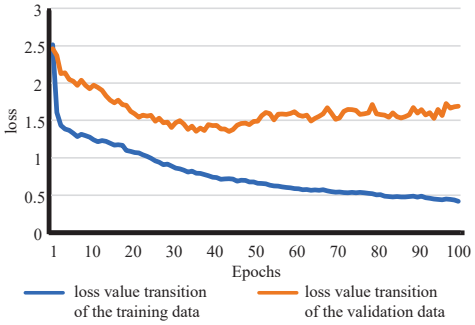


Fig. 10. Loss value for epochs in the Proposed Method 2.

The words in the expanded sentences generated by the RS task are identical to those in the original text. The expanded sentences generated by the RD task are also very similar to the original text. Because EDA applies tasks probabilistically, either the RS or RD task may be selected, potentially generating an expanded sentence that is exactly the same as the original sentence. As a result, overfitting caused by the expanded sentences generated by EDA leads to reduced accuracy. We propose methods to improve data expansion.

## IV. Proposed Method 2

### A. Method for Task Ratio Adjustment and Exclusion of Similar Extended Sentences

The RS task had a negative impact on learning structured specifications. We switched the RS task to the SR task effective for training data augmentation. Proposed Method 2 sets the task ratio of the RS task to 0%, and increases the task ratio of the SR task to 50% accordingly. The RI and RD tasks are executed at 25% task ratio, the same as Proposed Method 1.

Furthermore, to eliminate augmented sentences similar to the original sentences, we vectorized the original sentence and augmented sentences based on word frequency and calculated cosine similarity. For each sentence in the original text, we generated eight expanded sentences. One sentence is randomly selected from the expanded sentences within the cosine similarity $x$ range threshold and added to the training data. The threshold was set within the range $0.6 \leq x \leq 0.98$.

### B. Evaluation of Proposed Method 2

EDA adaptation rate $\alpha$ was set to 0.1. Other settings are the same as those for Proposed Method 1. Fig. 9 shows the results comparing Proposed Method 1 with CRF (81%) and the Attention seq2seq model (70%). The accuracy of Proposed Method 2 was 89%, surpassing the previous best accuracy achieved by CRF.

Fig. 10 shows the transition in loss values over epochs for the training data and validation data of Proposed Method 2. The loss value transition in the validation data shows a decreasing trend, unlike the proposed method 1. However, a slight upward trend appears to emerge after epoch 40. This suggests that while overfitting was resolved in the early epochs, it may have reoccurred after epoch 40.

## V. Conclusion

To improve accuracy beyond previous research, we proposed methods to increase training data through data augmentation in natural language processing. Simply applying EDA in Proposed Method 1 resulted in decreased accuracy. We proposed Proposed Method 2, which changes the RS task to the SR task and selects augmented sentences based on cosine similarity. This method achieved accuracy surpassing CRF, which had the highest accuracy in previous research. We also confirmed that overfitting was mitigated to some extent, demonstrating the effectiveness of the proposed methods.

Future research should investigate threshold settings (such as methods using confidence intervals) in Proposed Method 2. A method is needed to determine the optimal task ratio and EDA adaptation rate $\alpha$. Proposed Method 2 requires a strategy to address overfitting occurring after approximately 40 epochs. We need to conduct experiments with additional requirements to verify the proposed method's versatility. We need to research methods using Large Language Models (LLMs) and compare their accuracy. We aim to apply the proposed method to actual system development and verify its effectiveness.

## References

[1] Y. Fujita and K. Ueda, "A method for selecting training data using doc2vec for automatic test cases generation," in *2024 IEEE International Conference on Consumer Electronics*, Jan. 2024, pp. 1–6.

[2] T. Kudo, "Mecab: Yet another part-of-speech and morphological analyzer," Feb. 2013. [Online]. Available: http://taku910.github.io/mecab/

[3] ——, "Crf++: Yet another crf toolkit," Feb. 2013. [Online]. Available: http://taku910.github.io/crfpp//#source

[4] Y. Shimizu and K. Ueda, "Automatic test cases generation method by structuring requirement specification using lstm," in *2024 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan)*, 2024, pp. 453–454.

[5] ——, "Test case generation using an attention seq2seq model for structuring requirement specifications," *IEICE Communications Express*, vol. 14, no. 9, pp. 346–348, 2025.

[6] F. Bond, S. Wang, E. H. Gao, H. S. Mok, and J. Y. Tan, "Developing parallel sense-tagged corpora with wordnets," in *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*, A. Pareja-Lora, M. Liakata, and S. Dipper, Eds. Sofia, Bulgaria: Association for Computational Linguistics, Aug. 2013, pp. 149–158. [Online]. Available: https://aclanthology.org/W13-2319/